

SQL vs NoSQL

Databases are divided into Relational (SQL) and Non-Relational (NoSQL) systems. SQL (e.g., MySQL) uses structured tables and fixed schemas, while NoSQL (e.g., MongoDB) offers dynamic schemas and horizontal scaling for large-scale distributed apps.

DEFINITION:

SQL DATABASES

SQL databases are relational database management systems (RDBMS) that store data in structured tables (rows and columns).

- Data is organized using schemas
- Relationships are defined using foreign keys
- Queries are executed using SQL language

Examples:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

NoSQL DATABASES

NoSQL databases are non-relational and designed for flexible, distributed, and scalable data storage.

- Schema is dynamic or schema-less
- Data can be stored in multiple formats

Types of NoSQL databases:

- Document (MongoDB)
- Key-Value (Redis)
- Column-family (Cassandra)
- Graph (Neo4j)

KEY DIFFERENCES

Feature	SQL Databases	NoSQL Databases
Data Model	Structured (Tables/Rows)	Unstructured (Document, Key-Value, Graph)

Schema	Fixed schema	Dynamic / flexible schema
Query Language	SQL (standardized)	No standard query language
Scalability	Vertical scaling	Horizontal scaling
Consistency	Strong consistency (ACID)	Eventual consistency (BASE)
Transactions	Fully supported	Limited or eventual
Relationships	Supports joins and relations	Typically avoids joins
Performance	Slower for large distributed data	Faster for large-scale distributed systems
Use Case	Structured data & complex queries	Big data, real-time apps
Examples	MySQL, PostgreSQL	MongoDB, Cassandra

DATA STRUCTURE AND SCHEMA DESIGN

SQL

- Uses tables with predefined schema
- Each column has a data type
- Example:

```
CREATE TABLE Students (
  id INT,
  name VARCHAR(50),
  age INT
);
```

NoSQL

- Uses flexible formats like JSON documents

Example (MongoDB):

```
{
  "id": 1,
  "name": "Archana",
  "skills": ["Java", "ML"]
}
```

SCALABILITY

SQL (Vertical Scaling)

- Increase server capacity (CPU, RAM)
- Limited by hardware
- Expensive for large systems

NoSQL (Horizontal Scaling)

- Add more servers (distributed architecture)
- Supports sharding
- Ideal for cloud-based systems

CONSISTENCY MODEL

SQL - ACID Properties

- Atomicity
- Consistency
- Isolation
- Durability

Ensures:

- Reliable transactions
- No data inconsistency

NoSQL - BASE Model

- Basically Available
- Soft state
- Eventual consistency

Trade-off:

- Faster performance
- Slight delay in consistency

QUERYING CAPABILITIES

SQL

Powerful query capabilities:

- JOIN
- GROUP BY
- Aggregations

NoSQL

- Limited or no JOIN operations
- Queries depend on database type
- Focus on fast retrieval

USE CASE COMPARISON

When to Use SQL

- Banking systems
- Enterprise applications
- Inventory systems
- Applications requiring strict consistency

When to Use NoSQL

- Social media platforms
- Real-time analytics
- Big data applications
- IoT systems

ADVANTAGES AND DISADVANTAGES

SQL Advantages

- Structured and reliable
- Strong data integrity
- Standardized query language

SQL Disadvantages

- Less flexible
- Hard to scale horizontally

NoSQL Advantages

- Highly scalable
- Flexible schema
- Faster for large datasets

NoSQL Disadvantages

- Lack of standardization
- Weaker consistency
- Limited complex querying

KEY TRADE-OFF

NoSQL systems often prioritize: Availability + Partition Tolerance

SQL systems prioritize: Consistency + Reliability